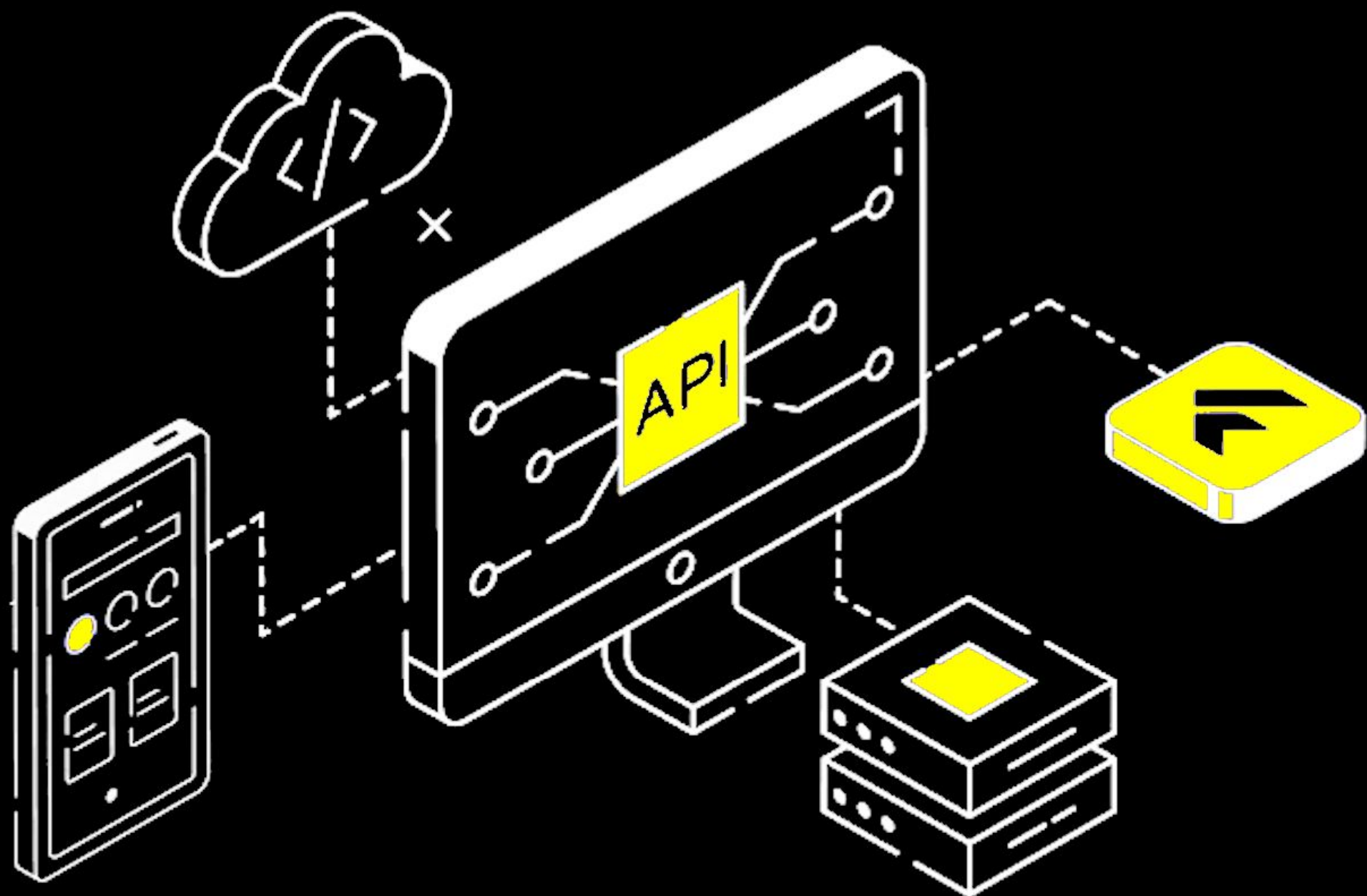




Flutter App Architecture

Mateusz Wojtczak

About me



Mateusz Wojtczak

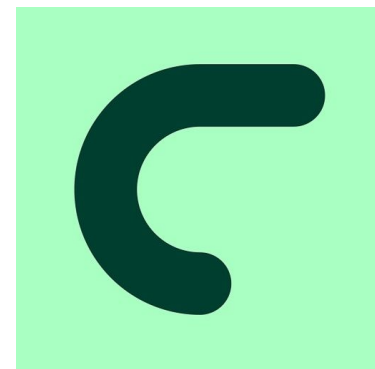
Head of Mobile @ LeanCode

Flutter & Dart GDE

@matiwojt

Over 40 Flutter apps

Startups



Virium

Scaleups



MMGtalent

INFINIT

Welliba



better with insight

kontomierz.pl



Quittercheck

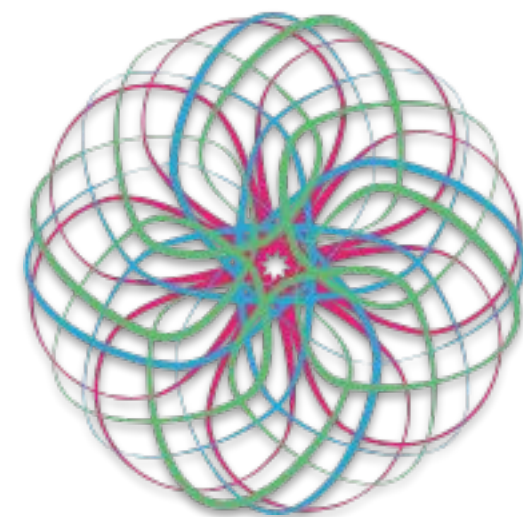
PRO\$PECTOR

STERYLIS®

NISZCZY ŹRÓDŁA INFЕКCJI



BIBLIA AUDIO
SUPERPRODUKCJA



CHDR

Centre for Human Drug Research

Enterprise



Banking app in Flutter in 2021...

...was a bold decision.

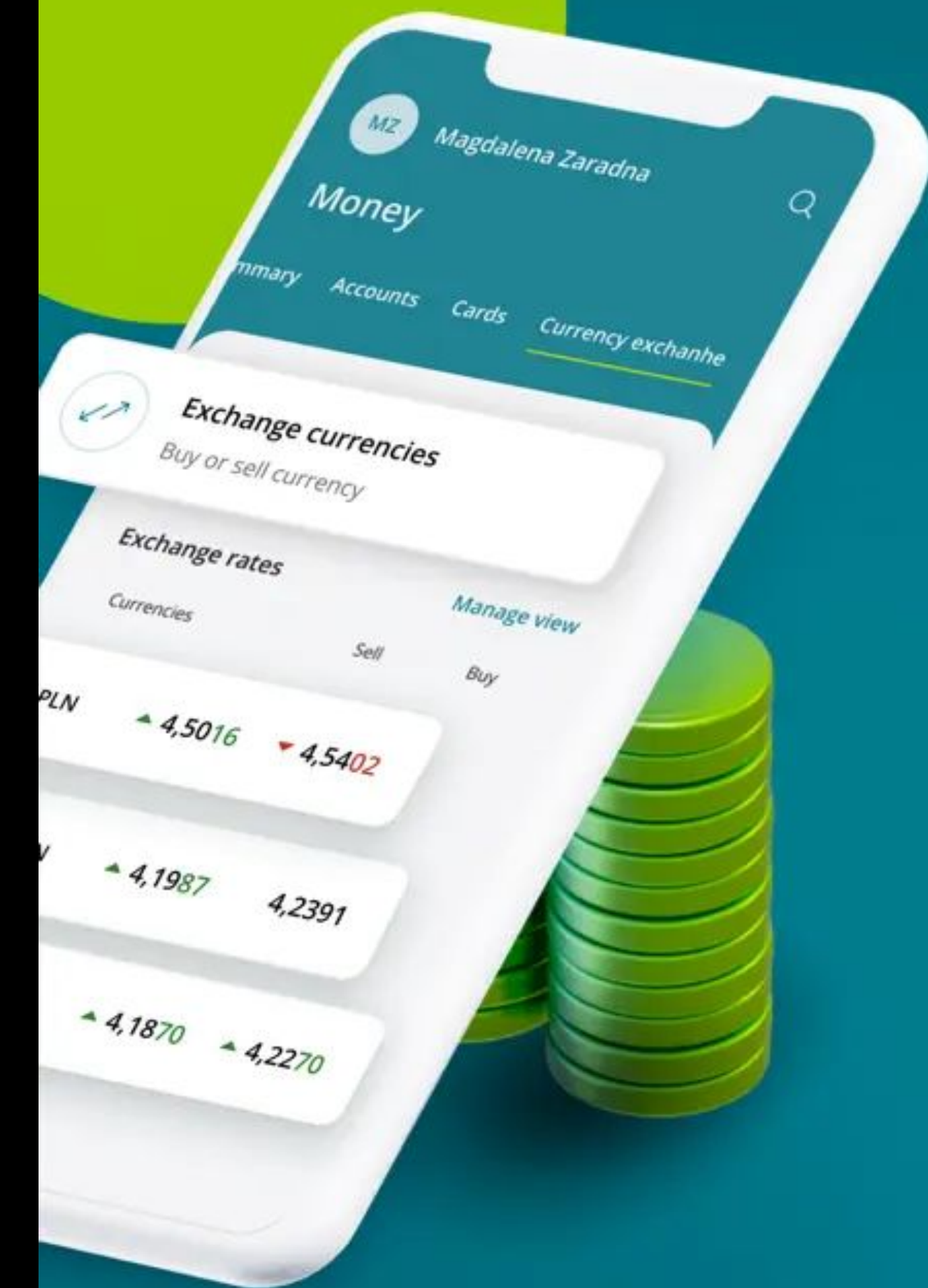
But it paid off

- 2021 Newsweek Ranking: places 10-20
- 2022 Newsweek Ranking: 10th place
- 2023 Newsweek Ranking: 7th place
- 2024 Newsweek Ranking: 3rd place 🕶️

CA24 Mobile 
- application
full of benefits



It was quite a journey.



**...and use
modern functionalities**

Battle-tested solutions

Our architecture

Project structure

State management

Dependency injection

API

Design system

Our architecture

- Project structure

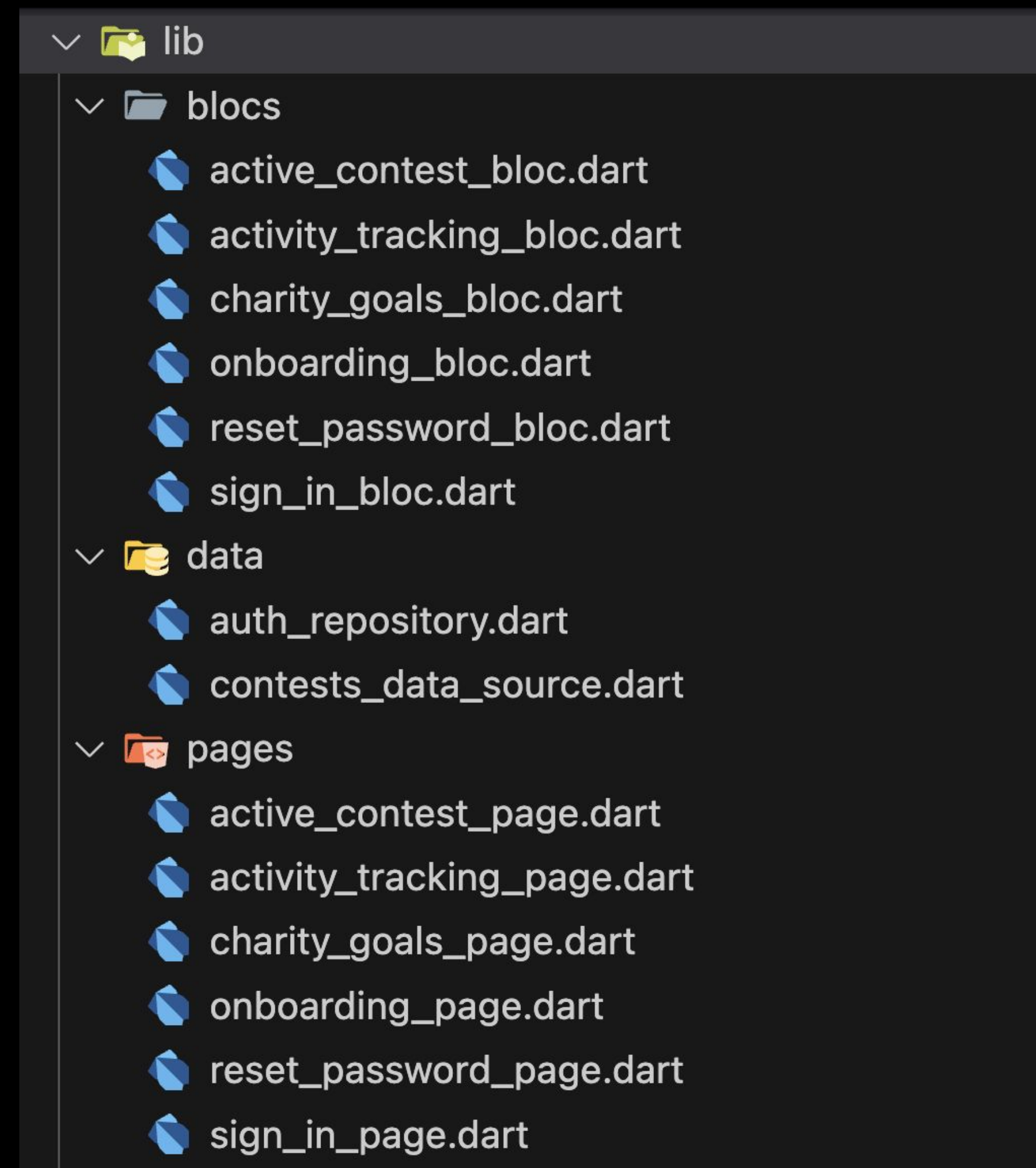
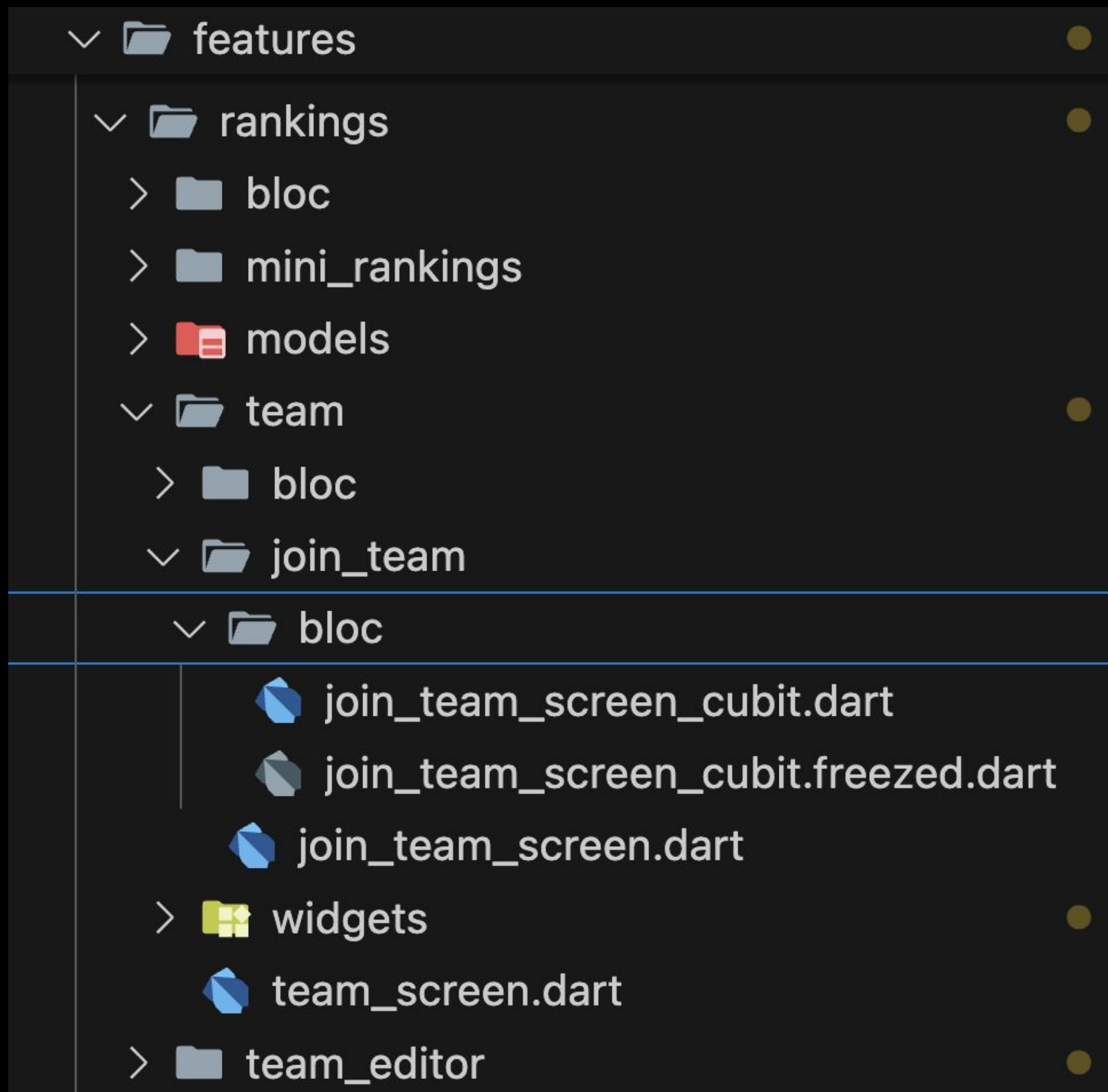
State management

Dependency injection

API

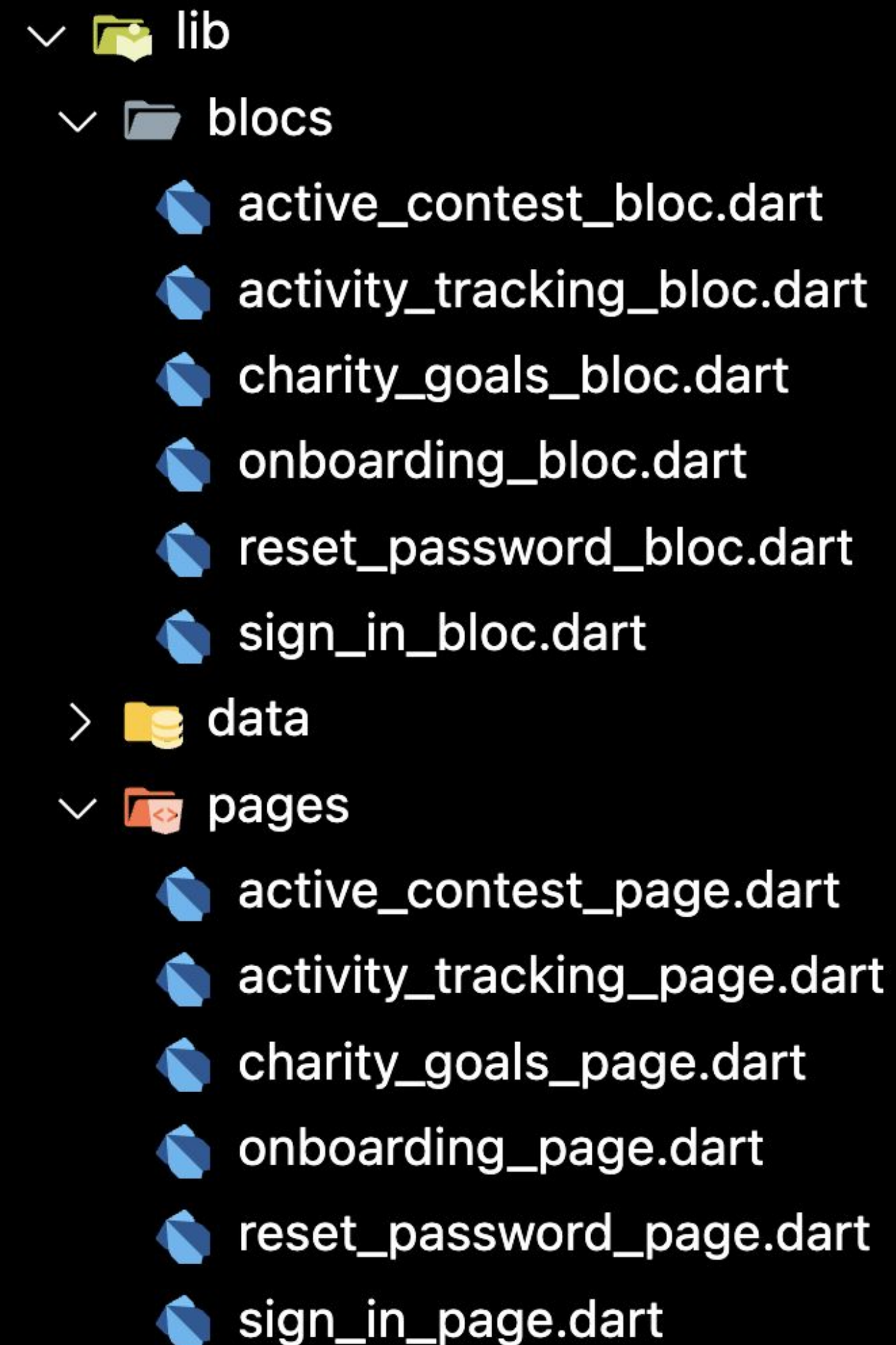
Design system

Feature-based architecture



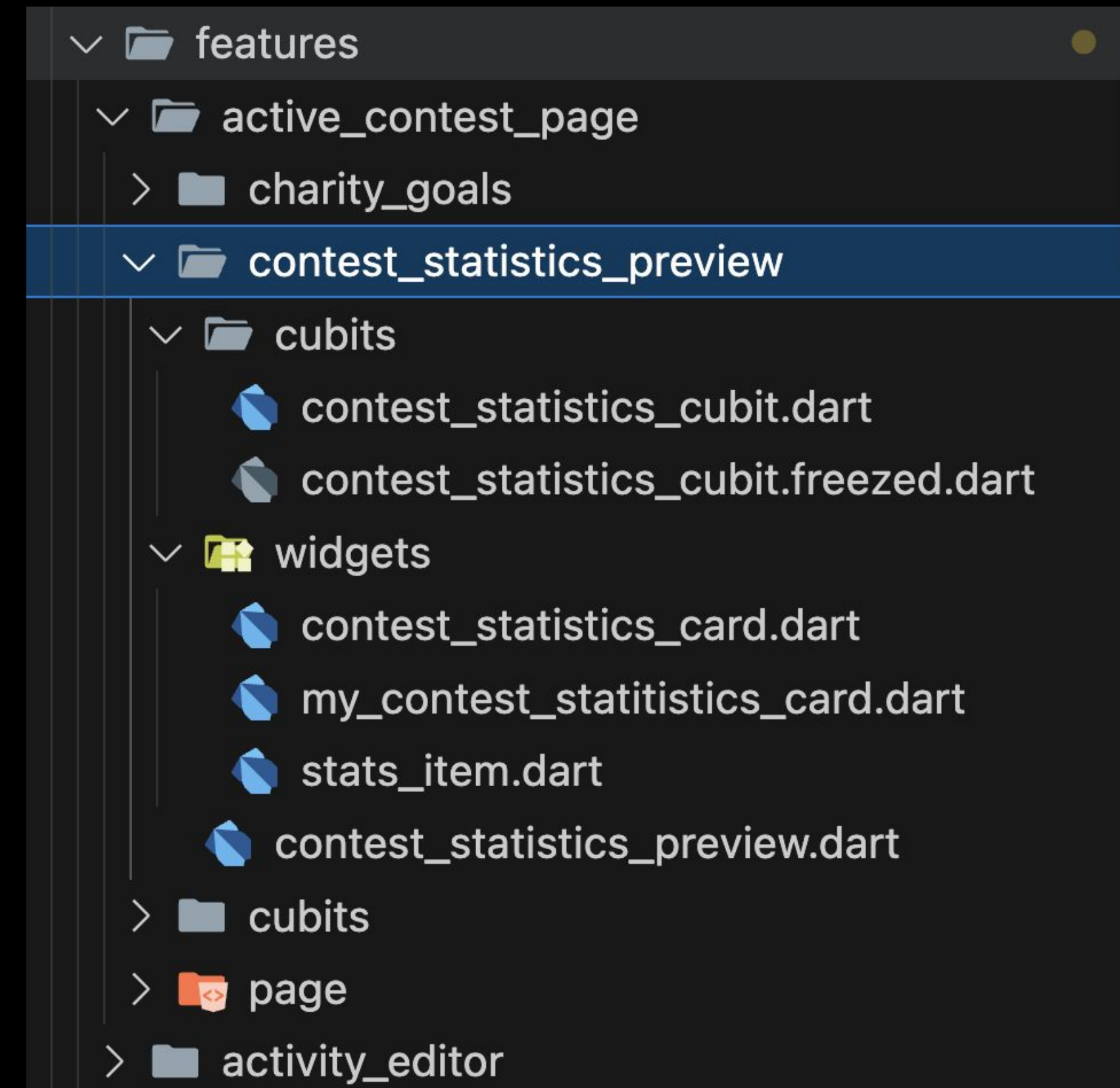
Layer-based architecture

- Hard to split into multiple packages
- You don't have to think where to put the file
- Scales really bad (imagine 100 blocs there)
- Hard to tell how big is the feature
- Hard to be new in the team



Feature-based architecture

- Domain-oriented
- Easy to encapsulate
- Easy to onboard new devs
- Convenient to add new features
- Easy to assess the complexity of a feature



Our architecture

Project structure

- State management

Dependency injection

API

Design system

Bloc is our default choice

Why Bloc?

- Simple and easy to understand principles
- Scalable
- Predictable and easy to test
- Can be boilerplate'y, but we built our tooling to reduce that
- Cubits are enough in over 95% cases

What if I need to navigate to another page?

What if I need to show a
snackbar?

What if I need to use context
inside my cubit?

```
void doSomething(Function navigate);
```

```
void doSomething(BuildContext context);
```

bloc_presentation

```
sealed class CommentCubitEvent {}

class FailedToUpvote implements CommentCubitEvent {
    const FailedToUpvote(this.reason);

    final String reason;
}
```

```
void upvote() {  
    // upvoting logic  
  
    if (!success) {  
        // we can emit it and forget about cleaning it from the state  
        emitPresentation(const FailedToUpvote('bad connection'));  
    } else {  
        emit(/* new state */);  
    }  
}
```

```
BlocPresentationListener<CommentCubit, CommentCubitEvent>(
  listener: (context, event) {
    switch (event) {
      case FailedToUpvote():
        ScaffoldMessenger.of(context)
          ..hideCurrentSnackBar()
          ..showSnackBar(SnackBar(content: Text(event.reason)));
    }
  },
  child: MyWidget(),
)
```

Hooks

```
final expanded = useState(false);  
final controller = useTextEditingController();  
final focusNode = useFocusNode();
```

```

class ProfilePage extends Page<void> {
  const ProfilePage();

  @override
  Route<void> createRoute(BuildContext context) => AppRoute(
    builder: (context) => BlocProvider(
      create: (context) => ProfileCubit(
        userRepository: context.read(),
      )..init(),
      child: const ProfileScreen(),
    ),
    settings: this,
  );
}

```

```

class ProfileScreen extends StatelessWidget {
  const ProfileScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return BlocBuilder<ProfileCubit, ProfileState>(
      builder: (context, state) => // ...
    );
  }
}

```

```

class ProfileScreen extends HookWidget {
  const ProfileScreen({super.key});

  @override
  Widget build(BuildContext context) {
    final cubit = useBloc(
      () => ProfileCubit(
        userRepository: context.read(),
      )..init(),
    );

    return BlocBuilder<ProfileCubit, ProfileState>(
      bloc: cubit,
      builder: (context, state) => // ...
    );
  }
}

```

```

class ProfilePage extends Page<void> {
  const ProfilePage();

  @override
  Route<void> createRoute(BuildContext context) => AppRoute(
    builder: (context) => BlocProvider(
      create: (context) => ProfileCubit(
        userRepository: context.read(),
      )..init(),
      child: const ProfileScreen(),
    ),
    settings: this,
  );
}

```

```

class ProfileScreen extends StatelessWidget {
  const ProfileScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return BlocBuilder<ProfileCubit, ProfileState>(
      builder: (context, state) => // ...
    );
  }
}

```

```

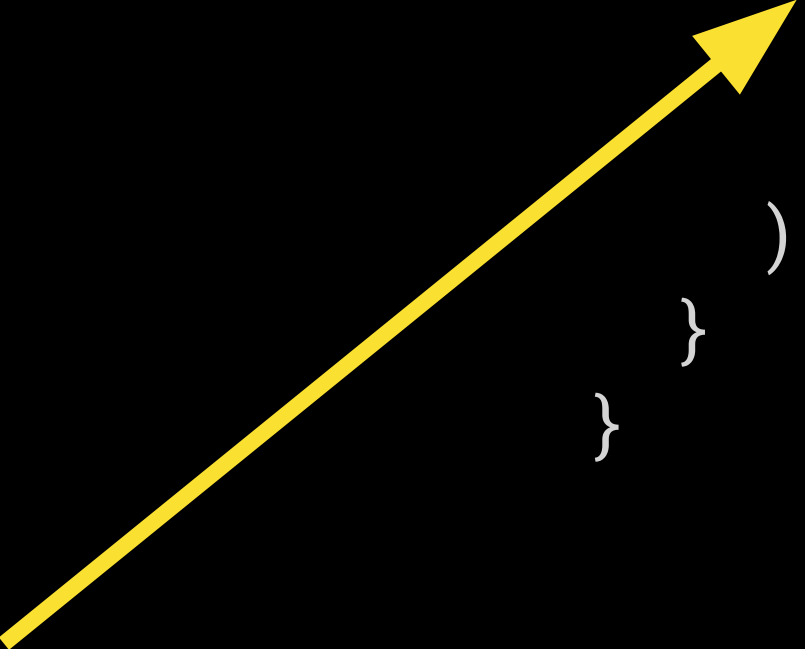
class ProfileScreen extends HookWidget {
  const ProfileScreen({super.key});

  @override
  Widget build(BuildContext context) {
    final cubit = useBloc(
      () => ProfileCubit(
        userRepository: context.read(),
      )..init(),
    );

    return BlocBuilder<ProfileCubit, ProfileState>(
      bloc: cubit,
      builder: (context, state) => // ...
    );
  }
}

```

Explicit dependency



```
@override  
Widget build(BuildContext context) {  
    useAutoRefreshProfileStatus();  
    // ...  
}
```

Hooks

- We use hooks by default
- Reduce boilerplate of StatefulWidget
- Make it easy to share lifecycle-dependent logic between widgets
- Custom lints to catch misuses (leancode_lint package)
- 🚨 Overuse of useEffect can cause widgets to become big and hard to understand

leancode_hooks 0.0.6

Published 9 months ago •  leancode.co Dart 3 compatible

SDK

FLUTTER

PLATFORM

ANDROID

IOS

LINUX

MACOS

WEB

WINDOWS

avoid_conditional_hooks

AVOID using hooks conditionally

BAD:

```
Widget build(BuildContext context) {  
  if (condition) {  
    useEffect(() {  
      // ...  
    }, []);  
  }  
}
```

BAD:

```
Widget build(BuildContext context) {  
  final controller = this.controller ?? useTextEditingController();  
}
```

BAD:

```
Widget build(BuildContext context) {  
  if (condition) {  
    return Text('Early return');  
  }  
  
  final state = useState(0);  
}
```

Our architecture

Project structure

State management

- Dependency injection

API

Design system

Provider + Bloc

- Binds a Bloc instance to some widget subtree lifecycle
- It gets disposed with the widgets
- Great for Blocs related to whole screens or features
- 🚧 Beware of deep Provider widget nesting

```
BlocProvider(  
  create: (context) => UserScopedStreamFeedCubit(  
    messagesRepository: context.read(),  
    authCubit: context.read(),  
    streamReactionsCubit: context.read(),  
  )..init(),  
),
```

get_it + your global stuff

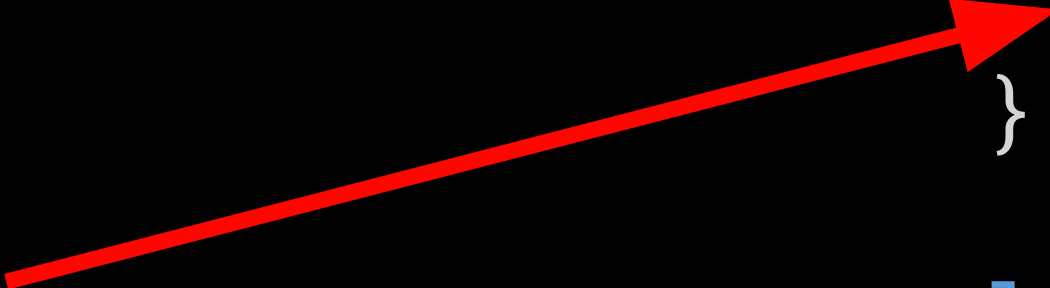
- A simple service locator
- Can be used without Flutter
- You don't have to think about your widget structure

```
void registerSingletons() {  
    GetIt.I.registerLazySingleton<ApiClient>(  
        () => ApiClient());  
    GetIt.I.registerLazySingleton<LocalStorage>(  
        () => LocalStorage());  
    GetIt.I.registerLazySingleton<SettingsCubit>(  
        () => SettingsCubit());  
}  
  
final apiClient = GetIt.I.get<ApiClient>();
```

✗ Don't resolve deps inside constructor

implicit dependency

```
class UserPreferencesCubit {  
    UserPreferencesCubit() {  
        _localStorage = GetIt.I.get<LocalStorage>();  
    }  
  
    late final LocalStorage _localStorage;  
}
```



Our architecture

Project structure

State management

Dependency injection

● API

Design system

Always generate API contracts



_profileApi.getP

 **getProfile(...)** → Future<Response<ProfileResponse>>

 **getProfileAddress(...)** → Future<Response<ProfileAddressRes...

 **getProfileAddresses(...)** → Future<Response<List<ProfileAdd...

 **getProfileContact(...)** → Future<Response<ProfileContactRes...

You can use OpenAPI / Swagger

Our architecture

Project structure

State management

Dependency injection

API

● Design system

Requirements

- Robust, highly reusable and adaptable set of UI components
- We don't want to be limited by what's in Material
- We don't want to create a text field from scratch in every project
- Avoid analyzing common UX patterns over and over again
- Good accessibility by default

Let's separate UX and UI

Search

AppTextFieldButton.ic...

context_menu

- AppContextMenu
 - AppContextMenu
 - AppContextMenu...
- AppContextMenuCo...
 - AppContextMenu...
- AppContextMenuItem
 - AppContextMenu...

icon

- AppIcon
 - AppIllustrationIc...
 - AppStandardIcons

navigation_bar

- AppNavigationBar
 - AppNavigationBar
 - AppNavigationBa...
- AppNavigationBarVi...
 - AppNavigationBa...
 - AppNavigationBa...

spinner

- AppIconSpinner
 - AppIconSpinner
- AppSpinner
 - AppSpinner

text_field

- AppTextField

AppTextField

AppTextField

Dialing Code

Prefix

Label

Suffix

Currency

Search

Hint text

Error message

Addons

Locale

en

Theme

Light

Text scale

1.00

Accessibility bold text

Text direction

Left-to-right

Knobs

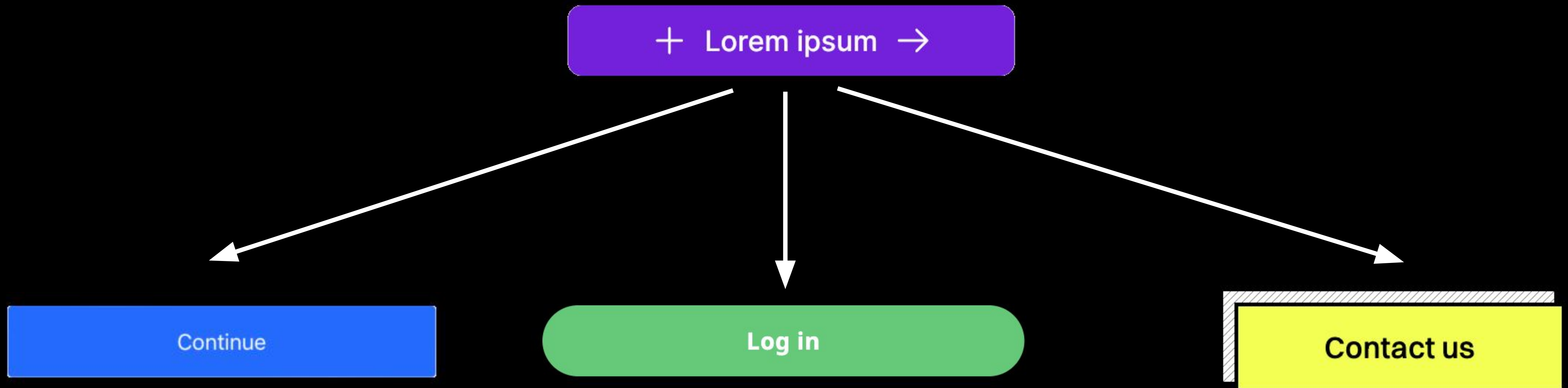
Enabled

Our design system template

- Separate Dart package
- Contains all common UI components
- Tries to handle many different variants
- Tested with widget and golden tests
- Supports assets compression with webp package
- Fetching icons from Figma file

This is not a new Material

You just copy the code from the template and change it as you wish (like shadcn/ui)



```

class OrdersPage extends Page<void> {
  const OrdersPage();

  @override
  Route<void> createRoute(BuildContext context) => AppRoute(
    id: PageId.orders,
    builder: (context) => const _OrdersScreen(),
    settings: this,
  );
}

class _OrdersScreen extends StatelessWidget {
  const _OrdersScreen({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    final s = 110;

    final cubit = OrderCubit();
    () => OrderCubit();

    return AppScreen(
      topBar: AppBar(
        title: s,
      ),
      body: PageView.builder(
        cubit: cubit,
        itemBuilder: (context, index) {
          final c = items[index];

          return Text(order.name);
        },
      ),
    );
  }
}

```

Text is forbidden within this design system.
 Use the alternative defined in the design system:
 AppText. dart(use_design_system_item_apptext)

```

(new) Text Text(
  String data, {
    Key? key,
    TextStyle? style,
    StrutStyle? strutStyle,
    TextAlign? textAlign,
    TextDirection? textDirection,
    Locale? locale,
    bool? softWrap,
    TextOverflow? overflow,
    double? textScaleFactor,
  },
)

```

leancode_lint 13.0.0

Published 20 days ago •  leancode.co Dart 3 compatible

[SDK](#) | [DART](#) [FLUTTER](#) | [PLATFORM](#) | [ANDROID](#) [IOS](#) [LINUX](#) [MACOS](#) [WINDOWS](#)

 52

[Readme](#) [Changelog](#) [Installing](#) [Versions](#) [Scores](#)

```
include: package:leancode_lint/analysis_options.yaml

custom_lint:
  rules:
    - use_design_system_item:
      AppTextField:
        - instead_of: TextField
          from_package: flutter
      AppColor:
        - instead_of: Color
          from_package: dart:ui
      AppColors:
        - instead_of: Colors
          from_package: flutter
        - instead_of: ColorScheme
          from_package: flutter
      AppText:
        - instead_of: Text
          from_package: flutter
      AppIcon:
        - instead_of: Icon
          from_package: flutter
    - prefix_widgets_returning_slivers:
      application_prefix: App
```

Building UI from ready-made blocks

```
return AppScaffold(  
  topBar: AppTopBar(  
    title: s.orders_title,  
  ),  
  body: Column(  
    children: [  
      AppTextField(  
        onChanged: cubit.updateSearchQuery,  
        decoration: AppTextFieldDecoration(label: s.common_search),  
      ),  
      Expanded(  
        child: PaginatedCubitLayout(  
          cubit: cubit,  
          itemBuilder: (context, data, index, items) {  
            final order = items[index];  
  
            return AppListItem(  
              title: order.name,  
              description: order.description,  
              icon: AppIcon(  
                AppIllustrationIcons.checkCircle,  
                color: context.colors.foregroundSuccessPrimary,  
                size: AppIllustrationIconSize.s32,  
                semanticsLabel: s.orders_checked_semantics_label,  
              ),  
            );  
          },  
        ),  
      ),  
    ],  
  ),  
);
```

Building UI from ready-made blocks

Wrapper which exposes limited and typed API

Only AppColors are allowed

Only predefined sizes are allowed

Semantics label is required

```
return AppScaffold(  
  topBar: AppTopBar(  
    title: s.orders_title,  
  ),  
  body: Column(  
    children: [  
      AppTextField(  
        onChanged: cubit.updateSearchQuery,  
        decoration: AppTextFieldDecoration(label: s.common_search),  
      ),  
      Expanded(  
        child: PaginatedCubitLayout(  
          cubit: cubit,  
          itemBuilder: (context, data, index, items) {  
            final order = items[index];  
  
            return AppListItem(  
              title: order.name,  
              description: order.description,  
              icon: AppIcon(  
                AppIllustrationIcons.checkCircle,  
                color: context.colors.foregroundSuccessPrimary,  
                size: AppIllustrationIconSize.s32,  
                semanticsLabel: s.orders_checked_semantics_label,  
              ),  
            ),  
          ],  
        ),  
      ),  
    ],  
  ),  
);
```

Building UI from ready-made blocks

- Keeping the design consistent
 - Strict API which doesn't allow passing arbitrary values
 - Linting
- Developers working on features don't need to deal with low-level UI stuff
 - This can be developed by dedicated developer specializing in UI
- UI polishing can be independently added later and is not blocking business features

What if we have a banking app
with 25+ devs working on it?

Flutter at scale - lessons learned

SQ01 - Overall Design

SQ02 - STP & Authentication

SQ03 - Product information, marketing & engagement

SQ04 - Onboarding

SQ05 - Accounts

SQ06 - Lending

SQ07 - Payments

SQ08 - VAS Basic

SQ09 - VAS CVP

SQ10 - Assisted Sales & Support

SQ13 - Platform & Framework

SQ15 - PFM & History

SQ17 - Investments & Insurance

SQ18 - Digital FX

Upgrade Flutter version every release

Tech debt accumulates much
faster in a large project

Enforce code ownership

Code ownership

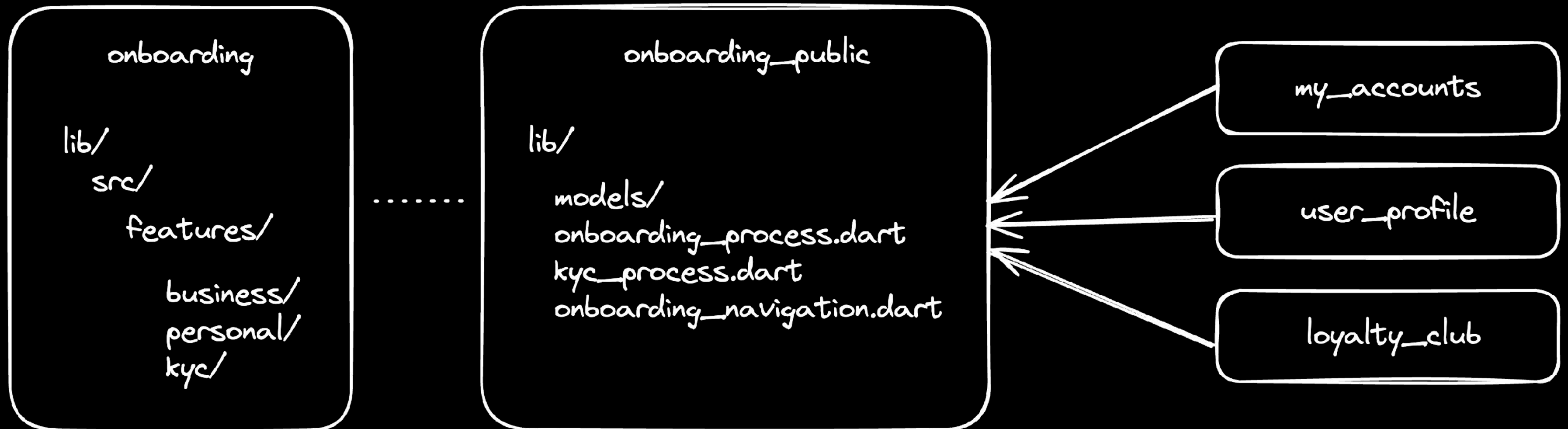
1. Every piece of code is owned by a team,
2. The team takes full responsibility of the code, meaning
 1. They will maintain it,
 2. They will develop it & fix bugs there,
 3. They will (dis)allow new code there,
3. There are no exceptions

Melos - task runner



- “A tool for managing Dart projects with multiple packages”
- CLI tool for mono-repos
- Filter packages that you want to do something with
- Popular users: FlutterFire, Flame, Flutter Community plus packages

Cross-squad communication



Testing policy

- Test all business logic (cubits and other domain classes)
- Test complex UI logic with widget tests
- Golden tests
 - Unit tests for design system components
 - Integration tests for whole screens
- We don't enforce any coverage thresholds

E2E testing

- We founded and develop Patrol
 - Overcoming limitations of `integration_test`
 - Native automation
 - Custom finders



Patrol

patrol.leancode.co



Banking transfer forms are hard

	A	B	C	D	E	F	G
3	Format enforcing (e.g. credit card number)	✓	?	✓ (focus flow between text fields)	✓	✓	?
4	Ready-to-use validators (e.g. only numbers, email)	✓	?	✓ (predefined validators)	✓	✓	✓
5	Allowing for custom validators	✓	✓	✓ (custom validators)	✓	✓	✓
6	Asynchronous validation - before submitting	?	✗	✓ (asynchronous validation)	✓	?	?
7	Mapping error when it comes from backend	?	✗	?	?	?	✓
8	Can we easily connect our custom widgets from design system?	✓	✓	✓	?	✓	✓
9	Data normalization when submitting (e.g. trimming)	✓	?	? (focus flow between text fields)	?	✓	?
10	Array fields (e.g. list of known languages)	✓	?	✓	?	✓	✓
11	Validation depending on another field's value	?	✗	✓	✓	✓	?
12	Focus: going to the next field / going to the next error / unfocus on submit	?	?	✓	?	?	?
13	When to validate (e.g. after every character)	?	✗	✓	?	✓	?
14	"Picking value for one or more fields can trigger a network request that can impact another fields value"	✓	✗	✓	?	?	?
15	scrollToFirstError - is it possible?	?	?	?	?	?	?

leancode_forms

- Getting rid of TextEditingController and FocusNodes
- Scroll to first error
- Conditional fields & fields depending on each other
- Async validation

leancode_forms 0.1.0 

Published 12 months ago •  [leancode.co](https://pub.dev/packages/leancode_forms) Dart 3 compatible

[SDK](#) | [FLUTTER](#) | [PLATFORM](#) | [ANDROID](#) | [IOS](#) | [LINUX](#) | [MACOS](#) | [WEB](#) | [WINDOWS](#)

There's more

bloc_presentation

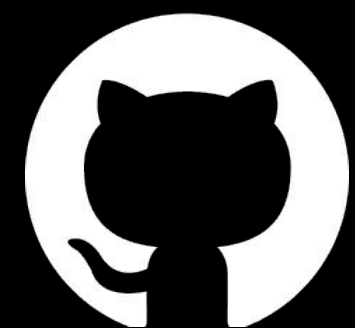
bloc_lens

patrol

poe2arb

arb_translate

mobile_tools



leancodep1



The first of its kind "Flutter CTO Report 2024"

▶ Scan it now



or go to leancode.co/flutter-report



Thank you

@matiwojt